

Aprendizaje Computacional

Dr. José Francisco Martínez Trinidad

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Introducción al Curso	2
2. Aprendizaje	4
3. Clasificadores lineales y perceptrón	7
4. Convergencia y generalización (perceptrón)	10
5. Máquinas de vectores de soporte (SVM)	14
6. SVM revisitada: forma dual y kernels	18
7. Preparación de datos y redes neuronales	24

1. Introducción al Curso

25 de agosto de 2026

Resumen rápido. Sesión inicial: contenido y dinámica del curso, temario, criterios de evaluación y el calendario del proyecto. La segunda mitad fue la introducción propiamente dicha: qué es aprender, por qué generalizar no es memorizar, de qué disciplinas se alimenta el área y la formalización de la función f que se quiere aprender contra la hipótesis h .

Info general

Contacto:

- Dr. José Francisco Martínez Trinidad
- Correo: fmartine@inaoep.mx

Temario:

1. Introducción y clasificadores lineales
2. Perceptrón
3. SVM
4. Clasificadores basados en distancia
5. ANNs
6. Evaluación, validación y overfitting
7. Métodos bayesianos
8. Árboles de decisión
9. Reglas de clasificación
10. Reglas de asociación
11. Selección de atributos
12. Clustering (K-means, gaussian mixture, jerárquicos)

Idea clave

No se trata de usar las herramientas como una caja negra. El objetivo es, como investigadores, saber cómo usarlas para generar nuevo conocimiento, y por eso hay que conocerlas a detalle: para poder innovar y proponer alternativas.

Evaluación

80 % proyecto:

- Equipo de dos personas
- Seleccionar un artículo reciente
- Proponer una solución alternativa
- Tres presentaciones

- Reporte final

20 % examen oral, a la entrega del proyecto, sobre un tema aleatorio del temario.

Calendario

- **25 de agosto.** Buscar un artículo publicado en 2026 que sea de interés, en una revista JCR.
- **3 de septiembre.** Entregar por correo una lista que indique cuál artículo seleccionó cada estudiante o equipo.
- **29 de septiembre o 1 de octubre.** Primera presentación:
 - Descripción del problema abordado en el artículo
 - Breve descripción del trabajo relacionado que menciona el artículo
 - Breve descripción de la solución propuesta en el artículo
 - Descripción de los experimentos que soportan dicha solución, incluyendo tablas y gráficas tomadas del artículo
 - Conclusiones a las que llegan los autores
 - Análisis de debilidades y posibles modificaciones para mejorar la propuesta de los autores
- **27 o 29 de octubre.** Segunda presentación: resumen del problema abordado.
- **24 o 26 de noviembre.** Presentación final: resumen del problema abordado y de la solución final.
- **30 de noviembre.** Entrega del reporte final. Se entrega en forma de artículo, con formato de <https://www.springer.com/>.

El reporte debe tener más o menos 12 páginas e incluir título, resumen, introducción, trabajo relacionado, solución propuesta, experimentos, conclusiones y trabajo futuro, y referencias.

Revistas JCR

- Expert Systems with Applications
- Knowledge-Based Systems
- Journal of Machine Learning Research

Buscador: <https://www.webofscience.com/wos/woscc/basic-research>

Recursos

- Weka 3
- scikit-learn

Ojito...

Hay que tratar de contactar al autor del artículo para que nos dé sus recursos y poder compararnos.

2. Aprendizaje

Que es el aprendizaje?

Es una disciplina dentro de las ciencias de la computación que surge como motivación de hacer que las computadoras realicen tareas complejas como las que hacen los humanos.

Aprender significa adquirir conocimiento, entendimiento o una habilidad de manera empírica o por estudio. La pregunta que queda abierta es cómo determinamos el éxito o el fracaso de la tarea.

Se busca que se logre un proceso de aprendizaje para que el sistema sea capaz de generalizar.

Idea clave

Generalizar no es memorizar. Aprender permite responder a problemas nuevos con el conocimiento adquirido.

Al inicio hubo varias tendencias. Una de ellas fue ver cómo aprenden los animales y de ahí trasladarlo a la máquina, con la premisa de que quizá el aprendizaje en un animal es diferente al de un ser humano, pero no resultó cierto.

Ojito...

No hay una demostración universal de lo que es el aprendizaje. Hay muchas definiciones y muchas escuelas.

Tipos de aprendizaje

- Supervisado
- No supervisado
- Por refuerzo

Tareas relacionadas con el aprendizaje

- Reconocimiento
- Diagnóstico
- Planeación
- Control de robots
- Predicción

Aprender puede ser percibir cambios en el comportamiento gracias a la experiencia. En nuestro contexto serían mejoras en actividades ya conocidas, y aprender tareas nuevas.

Hacer que las máquinas aprendan es muy útil, ya que nos simplifica la vida en muchos contextos: nos ayuda a hacer tareas complejas más rápido y a analizar datos masivos de una manera rápida.

Disciplinas relacionadas

- Estadística
- Lógica

- Matemáticas discretas
- Y muchas más

Todas las teorías que hay sobre cómo funciona el cerebro han ayudado bastante, ya que se trata de emular lo que se cree que ocurre en el cerebro humano cuando ocurre el aprendizaje, y trasladarlo a la computadora.

Otra disciplina que se vincula mucho con el ML es el control, la teoría de control. Igual los modelos psicológicos y los modelos evolutivos, entre muchos más.

Qué es lo que se aprende?

Diferentes estructuras computacionales: funciones, programas lógicos, máquinas de estado finito, gramáticas y sistemas para la resolución de problemas.

La idea es que tengo mi problema y tengo algunos ejemplos de él. Le doy a la computadora los ejemplos y la computadora me da como resultado el programa que me ayuda a generalizar cómo son esos ejemplos, de tal manera que cuando llegue un dato nuevo pueda clasificarlo, identificarlo y reconocerlo correctamente.

Más formalmente, se tiene una función f , que es la que queremos aprender, y otra función h , la hipótesis. Ambas reciben un vector de valores con N componentes. Queremos encontrar la función h cuya salida sea $h(X)$, y que sea capaz de reconocer un dato nuevo.

Supervisado y no supervisado

Hay dos tipos de aprendizaje, bueno tres, pero estos son los más comunes.

- **Supervisado.** Los datos de entrenamiento están etiquetados. A la función h , que es la que voy a aprender, le voy a dar un dato x , y esa función tiene que ser capaz de reconocer la nueva entrada.
- **No supervisado.** Los datos de entrenamiento no están etiquetados, y el algoritmo encuentra la relación entre los datos.

Vector de entrada

Al vector de entrada se le llama patrón, feature vector, instancia o ejemplo. Sus componentes son las variables o características, y esas variables pueden tomar valores reales, discretos o categóricos.

La salida puede ser un valor real, y entonces se trata de un predictor, o discreto, y entonces se trata de un clasificador.

Modos de entrenamiento

Hay varios modos de entrenamiento, dependiendo de los datos:

- Por batch
- De manera incremental
- En línea

Evaluación

Cuando ya tengo mi función h , ¿cómo la evalúo? ¿Cómo sé que está bien? Para eso se necesita un conjunto de prueba, que nos ayuda a ver qué tan bien la función h realiza su tarea, midiendo el error de h sobre ese conjunto. El conjunto de prueba es una partición del dataset original.

3. Clasificadores lineales y perceptrón

27 de agosto de 2026

Resumen rápido. Retomamos la idea de aprender una función que generalice y la aterrizamos en un ejemplo de control de acceso por rostro. De ahí salieron los clasificadores lineales, las funciones de pérdida y el perceptrón como algoritmo para ajustar los parámetros.

Recap

ML se trata de encontrar una función que fitee unos datos de entrenamiento para que sea capaz de generalizar. El proceso es:

1. Suponemos una función f que modela lo que queremos
2. Construimos hipótesis
3. Queremos construir la función h que emule a f
4. Entrenamos a h para que sea capaz de generalizar
5. Testeamos

Ejemplo: control de acceso por rostro

Supongamos que queremos construir un sistema que permita pasar a personas cuyos rostros sean conocidos para entrar al instituto. Tenemos los datos de las caras, y una imagen es un arreglo de bits.

Tenemos que construir una función

$$f : \mathbb{R}^d \rightarrow \{-1, 1\}$$

Es discreta porque es una tarea de clasificación. El clasificador tendrá un conjunto de vectores de entrenamiento con etiquetas, y estos vectores nos ayudarán a entrenar la función. Entonces queremos construir la función solo con los datos de entrenamiento, o sea con nuestra experiencia empírica.

Una manera sería ver un píxel de la imagen, de tal forma que al estar presente acepta a esa persona y la rechaza en otro caso, pero esto tiene varias limitaciones. ¿La función es útil para otras imágenes que no estén en el dataset? ¿Es capaz de generalizar?

Idea clave

Lo que se trata es de encontrar un modelo, una función que nos resuelva esa tarea de aprendizaje. No queremos funciones que fiteen perfectamente al conjunto de datos, ni súper generales, queremos algo en medio.

Selección de modelo

Y cómo se encuentran clasificadores que generalicen bien. La clave es *restringir* el conjunto de funciones binarias que uno se permite considerar. Se busca una clase de funciones tal que, si una función de la clase funciona bien en el entrenamiento, es probable que también funcione bien en las imágenes que no se han visto.

Idea clave

El problema de selección de modelo. La clase correcta no puede ser demasiado grande, porque entonces contiene demasiadas funciones claramente distintas y una de ellas va a memorizar el entrenamiento por casualidad, como la del píxel único. Pero tampoco demasiado chica, porque entonces se corre el riesgo de no encontrar ninguna función que funcione bien ni siquiera en el entrenamiento. Encontrar esa clase es uno de los problemas centrales del aprendizaje automático.

Fijar la clase de funciones en los clasificadores lineales, que es lo que sigue, es exactamente tomar esa decisión.

Clasificadores lineales

Lo que hacemos es definir una función lineal:

$$f(x; \theta) = \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d) = \text{sign}(\theta^\top x)$$

Se toma el signo para nuestro problema porque solo estamos considerando el caso binario: acepta o rechaza la entrada al edificio.

El modelo encontrará una línea, un plano que permita separar a las personas que pueden o no entrar al edificio. Ese plano es justo el lugar donde $\theta^\top x = 0$, y θ es el vector normal a él, así que ajustar θ es girar y mover el plano.

El chiste es encontrar la función a partir de las entradas que tengo en el entrenamiento. Cuando no es posible construir la función, queremos entonces descubrir otra que cometa el menor error posible.

Ojito...

En nuestro problema el clasificador lineal no guarda info de los píxeles vecinos, solo evaluaría el píxel individual.

Funciones de pérdida

Mencionamos que queremos encontrar el error. Esto se mide con la diferencia del dato esperado con el dato obtenido, y hay diferentes medidas de error.

A las funciones de error se les llama **loss**, función de error o función de pérdida, y hacen lo que mencionábamos: dicen qué tanta diferencia hay entre el dato observado y el esperado. Son esas funciones las que se tratan de optimizar.

El perceptrón

Queremos encontrar un clasificador lineal que minimice el error, o sea encontrar θ que minimice el error de entrenamiento:

$$E(\theta) = \frac{1}{n} \sum_{i=1}^n [1 - \delta(y^{(i)}, f(x^{(i)}; \theta))]$$

Donde δ vale 1 si sus dos argumentos coinciden y 0 si no, así que $1 - \delta$ vale 1 exactamente cuando el ejemplo quedó mal clasificado. Dicho de otra forma, $E(\theta)$ es la fracción de ejemplos de entrenamiento en los que el modelo se equivoca.

¿Pero cómo construir ahora esa función lineal? ¿Cómo ajustar los parámetros? ¿Hay alguna manera razonable de setear los parámetros de θ ?

Una opción es ajustar incrementalmente cada parámetro para que veamos que está bien. Un algoritmo simple de este tipo es el **perceptron update rule**:

$$\text{si } y^{(i)}(\theta^\top x^{(i)}) \leq 0 \quad \text{entonces} \quad \theta' = \theta + y^{(i)}x^{(i)}$$

En otras palabras, los parámetros son actualizados solo si hay error, y estos updates tienden a corregir errores. En nuestro caso, la clasificación de caras, cuando el modelo hace un error el signo no coincide con el de la etiqueta.

Supongamos que hacemos un error, y que los parámetros actualizados están dados por θ' . Si consideramos clasificar la misma imagen después del update:

$$\begin{aligned} y^{(i)}(\theta'^\top x^{(i)}) &= y^{(i)}((\theta + y^{(i)}x^{(i)})^\top x^{(i)}) \\ &= y^{(i)}(\theta^\top x^{(i)}) + (y^{(i)})^2 \|x^{(i)}\|^2 \\ &= y^{(i)}(\theta^\top x^{(i)}) + \|x^{(i)}\|^2 \end{aligned}$$

Como $y^{(i)} \in \{-1, 1\}$ su cuadrado siempre es 1, y $\|x^{(i)}\|^2 > 0$. O sea que el update le suma algo positivo, empujando ese ejemplo hacia el lado correcto del plano. Eso es lo que quiere decir que los updates tienden a corregir errores.

Ojito...

Esto da origen al reconocedor del perceptrón, y este reconocedor es útil para problemas lineales: solo puede separar con una línea o un plano, pero jamás podrá separar datos que no sean linealmente separables.

4. Convergencia y generalización (perceptrón)

1 de septiembre de 2026

Resumen rápido. En esta sesión vimos la convergencia del perceptrón, y la definición formal, que era el margen geométrico y cerramos con la intro a SVM

Punto de partida y supuestos

Seguimos con clasificadores lineales que pasan por el origen:

$$f(x; \theta) = \text{sign}(\theta^\top x),$$

donde θ denota los parámetros que tenemos que estimar en base a los ejemplos de entrenamiento, con ejemplos x y etiquetas y

Sea k el número de parámetros actualizados y $\theta^{(k)}$ el vector de parámetros después de k actualizaciones. Cada vez que el modelo se equivoca hace un update con la regla del perceptrón, y ese update es un k más

Para poder demostrar algo hacen falta dos supuestos:

1. **Las normas están acotadas:** existe R tal que $\|x^{(t)}\| \leq R$ para todo ejemplo.
2. **El clasificador lineal existe:** hay un θ^* , el θ ideal, que separa los datos con margen. Formalmente,

$$y^{(t)}(\theta^*)^\top x^{(t)} \geq \gamma > 0 \quad \text{para todo } t,$$

donde γ es el margen del clasificador, que es lo que nos dice que lo hace bien

Ojito...

En el ejemplo de las imágenes de las caras reconocidas para entrar al edificio, el segundo supuesto es justamente suponer que ese clasificador existe. Si los datos no son linealmente separables, no hay θ^* y nada de lo que sigue aplica

Convergencia en un número finito de updates

La convergencia se demuestra combinando dos resultados. El primero dice que el producto interno con el ideal crece linealmente, y el segundo que la norma cuadrada crece a lo más linealmente. Tomamos $\theta^{(0)} = 0$.

Parte 1: el producto interno crece linealmente

Supongamos que el update k se hizo sobre el ejemplo $(x^{(t)}, y^{(t)})$, o sea $\theta^{(k)} = \theta^{(k-1)} + y^{(t)} x^{(t)}$. Entonces

$$\begin{aligned} (\theta^*)^\top \theta^{(k)} &= (\theta^*)^\top \theta^{(k-1)} + y^{(t)} (\theta^*)^\top x^{(t)} \\ &\geq (\theta^*)^\top \theta^{(k-1)} + \gamma, \end{aligned}$$

donde la desigualdad es el supuesto del margen. Cada update le suma al menos γ , así que aplicándolo k veces desde $\theta^{(0)} = 0$:

$$(\theta^*)^\top \theta^{(k)} \geq k\gamma.$$

Parte 2: la norma cuadrada crece a lo más linealmente

Con el mismo update,

$$\begin{aligned}\|\theta^{(k)}\|^2 &= \|\theta^{(k-1)} + y^{(t)} x^{(t)}\|^2 \\ &= \|\theta^{(k-1)}\|^2 + 2 y^{(t)} (\theta^{(k-1)})^\top x^{(t)} + (y^{(t)})^2 \|x^{(t)}\|^2 \\ &\leq \|\theta^{(k-1)}\|^2 + R^2.\end{aligned}$$

El término de en medio se va porque el update ocurre precisamente cuando el ejemplo estaba mal clasificado, o sea cuando $y^{(t)} (\theta^{(k-1)})^\top x^{(t)} \leq 0$. Y el último se acota porque $(y^{(t)})^2 = 1$ y $\|x^{(t)}\| \leq R$. Repitiendo k veces:

$$\|\theta^{(k)}\|^2 \leq k R^2$$

Combinación: qué pasa con el ángulo

Si combinamos esos dos resultados podemos ver el coseno del ángulo entre el vector de la actualización k y el ideal:

$$\cos(\theta^*, \theta^{(k)}) = \frac{(\theta^*)^\top \theta^{(k)}}{\|\theta^*\| \|\theta^{(k)}\|} \geq \frac{k \gamma}{\|\theta^*\| \sqrt{k} R} = \sqrt{k} \frac{\gamma}{\|\theta^*\| R}.$$

El numerador se acotó por abajo con la parte 1 y el denominador por arriba con la parte 2. Como el coseno está acotado por 1:

$$\sqrt{k} \frac{\gamma}{\|\theta^*\| R} \leq 1 \quad \implies \quad k \leq \frac{R^2 \|\theta^*\|^2}{\gamma^2}.$$

Idea clave

Cada update acerca el coseno a 1 en una cantidad finita, o sea acerca el vector actual al ideal. Como el coseno no puede pasar de 1, solo se puede hacer un número finito de updates. Ahí está la convergencia

Ojito...

Está acotado, pero la cota depende del valor de los pesos: puede ser muy grande o muy pequeña, y de antemano no sabemos cuál.

Margen geométrico

El cociente $\|\theta^*\|^2 / \gamma^2$ nos dice qué tan difícil es el problema, o sea cuántos updates voy a tener que hacer. Su inverso es la distancia más chica que hay entre una instancia del conjunto de entrenamiento y mi región de decisión

Si esa distancia es grande hay bastante separación entre la superficie que divide las clases y las instancias, y es más fácil clasificar. Si es chica, es más difícil

Cómo se calcula

Se mide la distancia de la superficie de decisión a uno de los datos de entrenamiento. Sea $\tilde{x}$ el punto de la frontera más cercano a $x^{(t)}$, y ε esa distancia. Moverse de $x^{(t)}$ a la frontera es avanzar ε en la dirección normal, que es la de θ^* :

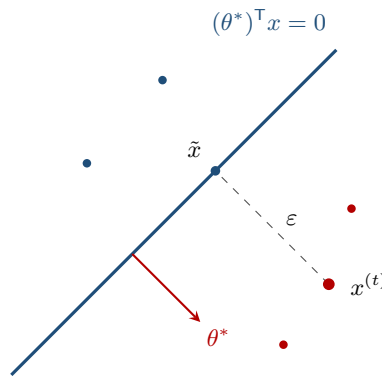
$$\tilde{x} = x^{(t)} - y^{(t)} \varepsilon \frac{\theta^*}{\|\theta^*\|}.$$

Evaluando el clasificador ideal en ese punto:

$$y^{(t)} (\theta^*)^\top \tilde{x} = \underbrace{y^{(t)} (\theta^*)^\top x^{(t)}}_{=\gamma} - \varepsilon \|\theta^*\| = \gamma - \varepsilon \|\theta^*\| = 0,$$

porque $\tilde{x}$ está sobre la frontera y ahí el clasificador vale cero. Despejando queda el **margen geométrico**:

$$\varepsilon = \frac{\gamma}{\|\theta^*\|}.$$



Por lo tanto el k , el número de veces que tengo que actualizar, se puede expresar en términos del margen geométrico:

$$k \leq \frac{R^2}{\varepsilon^2}.$$

Idea clave

Entre más ancho el pasillo que separa las clases, menos updates necesita el perceptrón. La dificultad del problema no está en cuántos datos hay ni en cuántas dimensiones, sino en qué tan apretado quedó ese pasillo

Generalización

Hasta ahorita asumimos que el conjunto de datos era linealmente separable, pero qué pasa si ahora no trabajo con mi conjunto de entrenamiento sino con datos que están fuera de él

Cuando llegue una instancia nueva, el clasificador lineal tendrá que ser capaz de clasificarla: se pone en el plano y se ve si la superficie separadora la separa correctamente

La garantía formal

El argumento se puede hacer preciso. Supongamos que *todas* las imágenes y etiquetas que podríamos llegar a encontrar, no solo las del entrenamiento, cumplen los mismos dos supuestos: $\|x_t\| < R$, y existe un θ^* finito con $y_t(\theta^*)^\top x_t \geq \gamma$ para toda t . O sea que existe un clasificador lineal que sirve para todo el problema, no solo para la muestra.

Ahora imaginemos que las instancias llegan una por una, y que por cada una hacemos a lo más un update si la clasificamos mal, y seguimos adelante sin volver a verla. ¿Cuántos errores vamos a cometer en esa secuencia arbitraria e indefinida?

Idea clave

El mismo número: $k \leq (R/\gamma_{\text{geom}})^2$. La cota no cambia. Una vez cometidos esos errores, todas las instancias nuevas se clasifican correctamente. Eso es una garantía de generalización, y no salió de un supuesto estadístico sobre la distribución de los datos, sino de la geometría del problema.

Ojito...

Con un pero: el algoritmo necesita *saber* cuándo se equivocó, porque la cota está expresada en número de updates, y los updates se hacen sobre errores. En un escenario real donde nadie te dice la etiqueta verdadera, no hay quién dispare el update.

Idea clave

Lo que queremos ahora es un clasificador lineal que nos dé el máximo margen geométrico. De todas las superficies que separan bien el entrenamiento, la que deja el pasillo más ancho es la que tiene más holgura para los datos nuevos. Ese clasificador son las máquinas de vectores de soporte

5. Máquinas de vectores de soporte (SVM)

3 de septiembre de 2026

Resumen rápido. En esta clase vimos como funcionan los SVM

El problema

Se plantea el problema de encontrar la superficie de separación entre los datos de una muestra, de tal manera que el margen geométrico que vimos sea el máximo posible

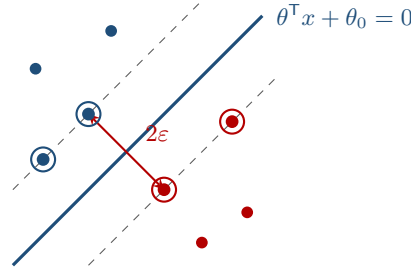


Figura 1: La superficie que deja el pasillo más ancho. Los puntos circulados, los que caen justo sobre el borde del margen, son los vectores de soporte.

Se plantea como un problema de optimización: lo que quiero optimizar son los parámetros de mi vector, de tal manera que exista un margen alrededor de mi superficie de separación que sea lo más grande posible.

Una manera de lograrlo sería encontrar una superficie que separe bien las instancias y luego tratar de que ese clasificador tenga el mayor margen posible. Se busca entonces maximizar el margen geométrico $\gamma / \|\theta\|$, cumpliendo las restricciones

$$y^{(t)}(\theta^T x^{(t)}) \geq \gamma \quad \text{para } t = 1, 2, \dots, n.$$

Idea clave

Esa restricción dice dos cosas al mismo tiempo. El *signo* de $y^{(t)}\theta^T x^{(t)}$ dice si la instancia quedó del lado correcto del plano, y su *magnitud* dice qué tan lejos quedó. Pedir que sea $\geq \gamma$ es pedir las dos: bien clasificada, y además a distancia de sobra, no pegada a la frontera.

Como problema de optimización

Maximizar el cociente que dijimos es lo mismo que minimizar su inverso al cuadrado:

$$\min_{\theta} \frac{1}{2} \frac{\|\theta\|^2}{\gamma^2} \quad \text{sujeto a} \quad y^{(t)}\theta^T x^{(t)} \geq \gamma, \quad t = 1, \dots, n.$$

Así escrito el problema depende de γ , y eso sobra. Si se escala θ y γ por la misma constante, ni las restricciones ni el margen geométrico cambian: la superficie es la misma. Se aprovecha esa libertad para fijar $\gamma = 1$, y el problema queda sin γ :

$$\min_{\theta} \frac{1}{2} \|\theta\|^2 \quad \text{sujeto a} \quad y^{(t)}\theta^T x^{(t)} \geq 1, \quad t = 1, \dots, n.$$

Con esa normalización el margen geométrico es $1/\|\theta\|$, así que minimizar la norma es literalmente ensanchar el pasillo.

El offset

Hasta ahora nuestra función está atada a pasar por el origen, y no queremos eso. Para lograrlo se suma un vector de offset θ_0 :

$$f(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0),$$

y el problema se vuelve

$$\min_{\theta, \theta_0} \frac{1}{2} \|\theta\|^2 \quad \text{sujeto a} \quad y^{(t)}(\theta^\top x^{(t)} + \theta_0) \geq 1.$$

Esto ayuda a que podamos encontrar una superficie con mayor margen geométrico, porque ya no obligamos al plano a pasar por el origen.

Vectores de soporte

Los elementos que caen justo a la distancia geométrica son los llamados **vectores de soporte**, y son elementos de la muestra. Son los que están circulados en la figura 1.

Propiedades del clasificador de margen máximo

A favor:

- **La solución es única** para cualquier conjunto de entrenamiento linealmente separable. El perceptrón, en cambio, se detiene en cualquier separador que funcione, y cuál depende del orden en que le llegaron los datos.
- **Es robusto a ejemplos ruidosos**, porque dejar la frontera lo más lejos posible de los datos da holgura para que una instancia corrida de lugar siga cayendo del lado correcto.
- **Depende solo de un subconjunto** de los ejemplos, los que caen exactamente sobre el margen. Los demás podrían estar en cualquier parte fuera del margen sin cambiar la solución.

En contra:

Ojito...

Es robusto a ejemplos ruidosos, pero **no a etiquetas ruidosas**. Un solo ejemplo mal etiquetado puede cambiar radicalmente el clasificador de margen máximo: si queda del lado equivocado, el pasillo tiene que encogerse hasta acomodarlo, y la frontera se va a otro lado. Esa fragilidad es la que motiva las variables de holgura.

Validación cruzada

Para evaluar este clasificador podemos hacer uso de la validación cruzada. Consiste en tomar el conjunto de entrenamiento, separar un cachito, construir el modelo con el resto y evaluar en ese cachito. El cachito se llama **fold**, y esto se repite k veces cambiando cuál fold se deja fuera.

Leave-one-out

Si dejamos solo un elemento fuera es el **leave-one-out cross validation**, pero eso se hace n veces, con n el número de elementos del training set. El error se mide así:

$$E_{\text{loo}} = \frac{1}{n} \sum_{i=1}^n \left[1 - \delta(y^{(i)}, f(x^{(i)}; \theta^{(-i)})) \right],$$

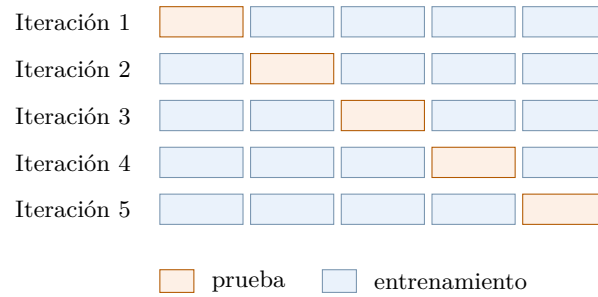


Figura 2: Validación cruzada con $k = 5$. En cada iteración un fold distinto hace de conjunto de prueba.

donde $\theta^{(-i)}$ son los parámetros entrenados con todo el conjunto *menos* el ejemplo i . O sea que cada ejemplo se predice con un modelo que nunca lo vio, y E_{loo} es la fracción de veces que ese modelo se equivoca.

Idea clave

El leave-one-out error está acotado por el número de vectores de soporte:

$$E_{\text{loo}} \leq \frac{\#\text{vectores de soporte}}{n}.$$

Cada vez que se quita una instancia que sea vector de soporte se puede cometer un error, porque quitarla mueve la superficie. Pero quitar una que no lo sea no cambia nada, así que ese ejemplo se sigue clasificando bien. Un clasificador con pocos vectores de soporte generaliza mejor.

Datos con ruido: variables de holgura

Uno de los problemas de este clasificador es que si las etiquetas del conjunto están mal, el clasificador va a estar incorrecto. También hay casos en los que el dato mismo trae error.

Para solucionar esto se introducen **variables de holgura**, dando una optimización relajada:

$$\min_{\theta, \theta_0, \xi} \frac{1}{2} \|\theta\|^2 + C \sum_{t=1}^n \xi_t$$

sujeta a

$$y^{(t)}(\theta^\top x^{(t)} + \theta_0) \geq 1 - \xi_t, \quad \xi_t \geq 0, \quad t = 1, \dots, n.$$

Cada ξ_t mide cuánto se le perdona a la instancia t , y C es un parámetro de costo, o sea qué tanto le va a costar violar las restricciones.

Ojito...

Si el costo no es tan grande, le voy a dar mucho chance a que queden instancias dentro del margen geométrico. Y al revés: con C grande no se permiten violaciones, el modelo se aprieta contra el entrenamiento y vuelve a ser sensible al ruido.

Cuando los datos no son linealmente separables

Todo lo anterior es solo para instancias linealmente separables, pero qué pasa si las instancias no lo son. A alguien se le ocurrió llevar las instancias que no son linealmente separables a un espacio en el que sí lo sean.

Entonces, en lugar de trabajar con la instancia del vector original, se trabaja con su imagen $\varphi(x)$ en el nuevo espacio. Ahí se construye el clasificador lineal y ahí se obtiene la respuesta:

$$y^{(t)}(\theta^\top \varphi(x^{(t)}) + \theta_0) \geq 1 - \xi_t.$$

Idea clave

La superficie sigue siendo un plano, pero en el espacio nuevo. Al regresarla al espacio original se ve curva, y por eso el modelo puede separar datos que una recta no separaba.

6. SVM revisitada: forma dual y kernels

8 de septiembre de 2026

Resumen rápido. Retomamos el problema de mapear las instancias a un espacio donde sí sean separables, y vimos que el mapeo explícito es caro y normalmente ni siquiera lo conocemos. La salida es reescribir la SVM en su forma dual, donde los ejemplos aparecen solo dentro de productos internos, y ahí sustituir el producto por un kernel. Cerramos con cómo se extiende todo esto a más de dos clases.

El mapeo explícito y su costo

La idea de la clase pasada: si el conjunto no es linealmente separable en $\mathbb{R}^d$, se busca una función φ que lleve las instancias a un espacio de mayor dimensión donde sí lo sean,

$$\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^{\mathcal{F}}, \quad \mathcal{F} > d.$$

Ejemplo 6.1. De dos dimensiones a tres, elevando y multiplicando coordenadas:

$$[x_1, x_2] \mapsto \varphi([x_1, x_2]) = [x_1^2, x_2^2, x_1x_2].$$

El producto interno en el espacio nuevo queda

$$\varphi(x)^\top \varphi(z) = x_1^2 z_1^2 + x_2^2 z_2^2 + x_1 x_2 z_1 z_2.$$

Con tres dimensiones de entrada el mismo esquema da nueve coordenadas, todos los productos $x_i x_j$.

Ojito...

Aquí está el problema. Encontrar la φ adecuada no es trivial, y aunque se encuentre, calcular productos internos en $\mathbb{R}^{\mathcal{F}}$ puede salir carísimo: la dimensión crece rápido. Peor aún, en general *no conocemos* φ explícitamente.

La salida no es buscar mejores mapeos, sino reformular la SVM de modo que φ nunca aparezca sola, solo dentro de productos internos. Eso es la forma dual.

La forma dual

El problema en el espacio de características es el de siempre:

$$\min_{\theta, \theta_0} \frac{\|\theta\|^2}{2} \quad \text{sujeto a} \quad y_t(\theta^\top \varphi(x_t) + \theta_0) \geq 1, \quad t = 1, \dots, n.$$

Es convexo y con restricciones lineales, y ese tipo de problemas se puede pasar a su forma dual con multiplicadores de Lagrange. Se introduce un escalar no negativo α_t por cada restricción:

$$J(\theta, \theta_0; \alpha) = \frac{\|\theta\|^2}{2} - \sum_{t=1}^n \alpha_t [y_t(\theta^\top \varphi(x_t) + \theta_0) - 1].$$

Idea clave

Cada α_t es el precio de violar la restricción t . Si la restricción se cumple con holgura, al maximizar sobre α le conviene poner $\alpha_t = 0$: esa restricción no está apretando y no cobra nada. Solo las que se cumplen con igualdad, o sea las que tocan el margen, terminan con $\alpha_t > 0$.

Bajo las condiciones de optimalidad de Lagrange se pueden intercambiar el mínimo y el máximo sin cambiar la respuesta:

$$\underbrace{\min_{\theta, \theta_0} \max_{\alpha \geq 0} J(\theta, \theta_0; \alpha)}_{\text{forma primal}} = \underbrace{\max_{\alpha \geq 0} \min_{\theta, \theta_0} J(\theta, \theta_0; \alpha)}_{\text{forma dual}}.$$

Resolviendo primero el mínimo interno, o sea derivando e igualando a cero:

$$\begin{aligned} \frac{\partial J}{\partial \theta_0} &= - \sum_{t=1}^n \alpha_t y_t = 0, \\ \frac{\partial J}{\partial \theta} &= \theta - \sum_{t=1}^n \alpha_t y_t \varphi(x_t) = 0 \implies \theta = \sum_{t=1}^n \alpha_t y_t \varphi(x_t). \end{aligned}$$

Idea clave

Esa segunda ecuación dice algo fuerte: la solución θ siempre vive en el subespacio generado por los ejemplos de entrenamiento. No es un vector cualquiera de $\mathbb{R}^{\mathcal{F}}$, es una combinación lineal de las propias instancias, pesada por $\alpha_t y_t$.

Sustituyendo esa forma de θ en el objetivo se llega al dual:

$$\max_{\alpha} \sum_{t=1}^n \alpha_t - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j [\varphi(x_i)^{\top} \varphi(x_j)]$$

sujeto a

$$\alpha_t \geq 0, \quad \sum_{t=1}^n \alpha_t y_t = 0.$$

Sigue siendo un problema de programación cuadrática, pero las restricciones son mucho más simples que las del primal.

La matriz de Gram

Fíjate en lo que quedó: la dimensión $\mathcal{F}$ del espacio de características **ya no aparece**. El problema está formulado enteramente sobre la matriz de productos internos, llamada matriz de Gram:

$$K = \begin{bmatrix} \varphi(x_1)^{\top} \varphi(x_1) & \cdots & \varphi(x_1)^{\top} \varphi(x_n) \\ \vdots & \ddots & \vdots \\ \varphi(x_n)^{\top} \varphi(x_1) & \cdots & \varphi(x_n)^{\top} \varphi(x_n) \end{bmatrix}.$$

Es de $n \times n$, o sea que su tamaño depende del número de ejemplos, no de la dimensión del espacio. Ahí está la puerta.

El truco del kernel

Idea clave

El truco del kernel. Si tienes un algoritmo donde los ejemplos aparecen *solo* dentro de productos internos, puedes reemplazar ese producto interno por otro, sin siquiera conocer φ .

Se define entonces una función kernel $K(x, x')$ tal que

$$K(x, x') = \varphi(x)^\top \varphi(x'),$$

y se sustituye. Nunca se calcula $\varphi(x)$: basta con conocer la forma funcional del kernel. Lo que costaba $\mathcal{F}$ operaciones ahora cuesta lo que cueste evaluar K , que suele ser $\mathcal{O}(d)$.

Teorema 6.2 (Mercer). *Para toda función $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ simétrica y semidefinida positiva, existen un espacio de Hilbert $\mathcal{F}$ y una función $\varphi : \mathbb{R}^d \rightarrow \mathcal{F}$ tales que*

$$K(x, x') = \langle \varphi(x), \varphi(x') \rangle.$$

Idea clave

Este teorema es el permiso para trabajar al revés. No hace falta construir el kernel a partir de un conjunto de funciones base $\varphi_1(x), \dots, \varphi_r(x)$: basta definir una K que cumpla las dos condiciones, simetría y semidefinida positiva, y el teorema garantiza que *alguna* φ existe. Nunca hay que verla.

Kernels estándar

Polinomial	$K(x, x') = (1 + x^\top x')^p, \quad p = 1, 2, \dots$
Base radial (RBF)	$K(x, x') = \exp\left(-\frac{\beta}{2} \ x - x'\ ^2\right), \quad \beta > 0$

Con $p = 1$ el polinomial es prácticamente el producto interno original, o sea la SVM lineal de siempre. El RBF corresponde a un espacio de características de dimensión infinita, y aun así se evalúa con una resta, una norma y una exponencial.

Clasificar con la solución dual

Ya con los α_t resueltos, una instancia nueva se clasifica con la función discriminante:

$$\hat{y}(x) = \theta^\top \varphi(x) + \theta_0 = \sum_{t \in \text{SV}} \alpha_t y_t K(x_t, x) + \theta_0,$$

donde SV es el conjunto de vectores de soporte, o sea los que tienen $\alpha_t \neq 0$.

Idea clave

Aquí reaparece la dispersión que ya conocíamos. Muchos α_t salen exactamente cero de la optimización, y son distintos de cero *solo* para los vectores de soporte. Por eso la suma corre sobre SV y no sobre los n ejemplos: los demás no aportan nada al clasificar.

Ojito...

No sabemos qué ejemplos van a ser vectores de soporte hasta haber resuelto la optimización. Y su identidad depende del kernel: cambiar de kernel cambia quiénes son.

De dónde sale θ_0

El offset se cayó del problema dual, así que hay que recuperarlo aparte. Se usa que para todo vector de soporte la restricción se cumple con igualdad:

$$y_i \left(\sum_{t \in \text{SV}} \alpha_t y_t K(x_t, x_i) + \theta_0 \right) = 1, \quad i \in \text{SV},$$

y como $y_i \in \{-1, 1\}$ implica $y_i^2 = 1$, se despeja

$$\theta_0 = y_i - \sum_{t \in \text{SV}} \alpha_t y_t K(x_t, x_i).$$

El margen geométrico en forma kernel

Sigue siendo $1/\|\theta\|$, solo que ahora $\|\theta\|$ se escribe con el kernel:

$$\hat{\gamma}_{\text{geom}} = \left(\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \right)^{-1/2}.$$

La forma dual relajada

Todo lo anterior supone que los datos sí son separables en el espacio de características. Con variables de holgura el primal era

$$\min \frac{\|\theta\|^2}{2} + C \sum_{t=1}^n \xi_t \quad \text{sujeto a} \quad y_t(\theta^\top \varphi(x_t) + \theta_0) \geq 1 - \xi_t.$$

Su dual es casi idéntico al de arriba. La *única* diferencia es que los multiplicadores quedan acotados por arriba:

$$\max_{\alpha} \sum_{t=1}^n \alpha_t - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{sujeto a} \quad 0 \leq \alpha_t \leq C, \quad \sum_{t=1}^n \alpha_t y_t = 0.$$

Idea clave

Por qué la cota superior: sin ella, una restricción imposible de satisfacer haría crecer su α_t hasta infinito, porque el precio de violarla nunca dejaría de subir. El valor C es justamente el punto donde uno decide dejar de insistir con esa restricción. Es el mismo C del primal, visto desde el otro lado.

Ojito...

Con holgura, θ_0 ya no se despeja con cualquier $\alpha_t > 0$. Hay que usar los que cumplen $0 < \alpha_t < C$, o sea excluir los que están en el tope. Un $\alpha_t = C$ marca un punto que *viola* la restricción de margen, y para ese la igualdad no se cumple.

Valor de α_t	Dónde cae el ejemplo	Sirve para θ_0
$\alpha_t = 0$	Fuera del margen, bien clasificado	No
$0 < \alpha_t < C$	Justo sobre el margen	Sí
$\alpha_t = C$	Viola el margen	No

Clasificación multiclase

Hasta aquí todo fue binario. En multiclase cada punto de entrenamiento pertenece a una de N clases distintas, y se quiere una función que, dado un punto nuevo, determine correctamente a cuál pertenece.

Ojito...

Eso **no** es lo mismo que multitiqueta. Hay escenarios con varias categorías donde un punto puede pertenecer a varias a la vez, y ese es otro problema.

One vs All (OVA)

Se construyen N clasificadores binarios. Para el i -ésimo, los positivos son todos los puntos de la clase i y los negativos todos los que no. Si f_i es ese clasificador, se decide con

$$f(x) = \arg \max_i f_i(x).$$

All vs All (AVA)

Un clasificador por cada par de clases i, j , donde i son los positivos y j los negativos. Como $f_{ji} = -f_{ij}$, en realidad solo hay que entrenar $N(N-1)/2$ de ellos. Se decide sumando votos:

$$f(x) = \arg \max_i \sum_j f_{ij}(x).$$

También se le llama *all-pairs* o *one-vs-one*.

Idea clave

OVA y AVA son tan simples que mucha gente los inventó por separado. Y eligiendo buenos clasificadores binarios de base, funcionan tan bien como cualquier otra cosa que se te ocurra.

	OVA	AVA
Clasificadores	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$
Tamaño de cada uno	Todo el conjunto	Solo dos clases
Balance de clases	Desbalanceado	Balanceado

AVA sale más rápido y usa menos memoria pese a entrenar más clasificadores, porque cada uno es mucho más chico. Si el tiempo de entrenamiento es superlineal en el número de puntos, que es el caso de la SVM, AVA es la mejor opción. OVA además produce muestras de entrenamiento desbalanceadas: una clase contra las otras $N-1$ juntas.

SVM multiclase directa

En lugar de armar varios clasificadores binarios, lo natural es distinguir todas las clases en una sola optimización.

Weston y Watkins (1999). Un vector de pesos w_m por clase, y una holgura $\xi_{i,t}$ por cada ejemplo y cada clase equivocada:

$$\min \quad \frac{1}{2} \sum_{m=1}^k w_m^\top w_m + C \sum_{i=1}^l \sum_{t \neq y_i} \xi_{i,t}$$

sujeto a

$$w_{y_i}^\top \varphi(x_i) + b_{y_i} \geq w_t^\top \varphi(x_i) + b_t + 2 - \xi_{i,t}, \quad \xi_{i,t} \geq 0.$$

Se decide con $\arg \max_m (w_m^\top \varphi(x) + b_m)$. Su desventaja principal es el tiempo de cómputo: el problema cuadrático resultante es enorme.

Crammer y Singer (2001). Misma idea pero con una sola holgura ξ_i por ejemplo, no una por clase:

$$\min \frac{1}{2} \sum_{m=1}^k w_m^\top w_m + C \sum_{i=1}^l \xi_i$$

sujeto a

$$w_{y_i}^\top \varphi(x_i) - w_t^\top \varphi(x_i) \geq 1 - \delta_{y_i, t} - \xi_i,$$

donde $\delta_{i,j}$ es la delta de Kronecker, que vale 1 si $i = j$ y 0 si no. La optimización se hace sin los términos de offset b_m .

SimMSVM (He et al., 2012). Reduce el número de restricciones de margen a l , una por ejemplo, comparando la clase correcta contra el promedio de las demás:

$$w_{y_i}^\top \varphi(x_i) - \frac{1}{k-1} \sum_{m \neq y_i} w_m^\top \varphi(x_i) \geq 1 - \xi_i.$$

Sus autores probaron que su formulación dual es exactamente la de Crammer y Singer con una restricción adicional, y cada modelo de clase se construye sobre sus propios vectores de soporte.

7. Preparación de datos y redes neuronales

10 de septiembre de 2026

Resumen rápido. Antes de aplicar cualquier clasificador hay que preparar los datos: limpiarlos y ponerlos en una escala común, decidiendo entre normalizar y estandarizar. Luego, cómo partir la muestra para entrenar sin caer en overfitting ni en underfitting, y la variante estratificada de la validación cruzada para datos desbalanceados. Al final se abrió el tema nuevo, las redes neuronales artificiales, con la neurona biológica que les sirve de modelo.

Preparación de los datos

A veces queremos aplicar un clasificador, un algoritmo que nos permita separar, clasificar o estimar. Pero antes están los datos, y lo primero es limpiarlos: eliminar columnas vacías, llenar los valores faltantes y demás.

Después viene la escala. Los atributos suelen venir en unidades distintas, y para evitar que unos dominen a otros hay que **normalizar** o **estandarizar**.

Técnica	Qué hace	Fórmula
Estandarizar	Media 0, varianza 1	$z = \frac{x - \mu}{\sigma}$
Normalizar	Todo entre 0 y 1	$x' = \frac{x - \text{mín}}{\text{máx} - \text{mín}}$

Estandarizar reordena los datos en función de su media y su varianza. Eso uniforma el manejo, para que ninguna variable se salga del rango de las demás, y de paso aplasta a los datos atípicos: reduce esa diferencia en vez de dejar que arrastre todo.

Escoger entre las dos es una de las decisiones que hay que tomar, y depende de los datos y del objetivo. La regla que se dio en clase:

Distribución	Qué aplicar
No normal	Estandarizar
Normal	Normalizar

Idea clave

La razón de fondo es de dónde salen las constantes de cada fórmula. Normalizar usa el mínimo y el máximo, que son los dos estadísticos más sensibles que existen a un valor extremo: si la distribución tiene cola larga, un solo punto lejano estira el denominador y comprime a todos los demás en una franja diminuta. Estandarizar usa media y desviación, que no se dejan mover tanto por un punto. Por eso la distribución no normal pide estandarizar. Si la distribución es normal, el mínimo y el máximo se portan bien y normalizar es seguro.

Ojito...

El escalado se aplica **por variable**, no sobre todos los datos juntos. Cada columna tiene su propia media y desviación, o su propio mínimo y máximo. Mezclarlas destruye justo lo que se quería arreglar.

Todo esto sirve para que en el clasificador lineal no haya descompensaciones en los pesos. Si un atributo va de 0 a 1 y otro de 0 a 100 000, el peso del segundo tiene que volverse diminuto para compensar, y el entrenamiento se vuelve lento y mal condicionado.

Cómo dividir el conjunto de muestra

La siguiente decisión es cómo partir la muestra para el proceso de entrenamiento. Los dos extremos fallan por razones opuestas:

- Si le pasamos **todos** los datos al clasificador, se los va a aprender y no va a poder generalizar. Eso es **overfitting**, y además nos quedamos sin manera de medirlo, porque ya no hay datos que el modelo no haya visto.
- Si le pasamos **muy pocos**, no alcanza a aprender el patrón de manera adecuada. Eso es **underfitting**.

De ahí las tres particiones, cada una con su trabajo:

Partición	Para qué
Entrenamiento	Ajustar los parámetros del modelo
Validación	Escoger el modelo y sus hiperparámetros
Prueba	Estimar el error real, una sola vez

Los repartos usuales son 70/15/15 o 80/10/10 por ciento.

Ojito...

El conjunto de prueba se toca una vez, al final. Si se usa para decidir algo, deja de ser una estimación honesta del error: se convierte en parte del entrenamiento, y la generalización que reporta ya está contaminada.

Validación cruzada estratificada

La otra opción es la validación cruzada, que ya se vio en la clase del 3 de septiembre (figura 2): se parte el conjunto en k folds y se rota cuál se deja fuera.

Con datos desbalanceados eso no basta. Se parten **por clases** y los folds se arman con datos de cada clase, de modo que cada fold conserve la proporción original.

Idea clave

Sin estratificar, con una clase rara y folds aleatorios puede tocar un fold sin un solo ejemplo de esa clase. Ese fold no mide nada sobre ella, y el promedio de los k errores sale optimista por una razón que no tiene que ver con el clasificador.

Redes neuronales artificiales

Hasta aquí todos los modelos salieron de plantear un problema de optimización. Pero alguien preguntó por qué no tratamos de emular el modo en que aprende el cerebro humano, y de ahí nace la idea de las redes neuronales artificiales.

La neurona biológica

El cerebro está conectado por neuronas. Las conexiones entre neuronas se llaman **sinapsis**, y se dan entre axones y dendritas.

Parte	Qué hace
Dendritas	Reciben las señales de otras neuronas
Soma	Integra lo que llega y decide si dispara
Axón	Lleva la señal de salida hacia afuera
Sinapsis	Une el axón de una con la dendrita de otra, y es donde la señal se pondera

Por qué es eficiente el cerebro? Por el número: del orden de 10^{11} neuronas y unas 10^{14} conexiones entre ellas. No es que cada neurona sea potente, es que son muchísimas y están densamente conectadas.

La neurona artificial

El modelo artificial se queda con la estructura y tira la biología. Cada parte tiene su contraparte:

Biológica	Artificial
Dendritas	Las entradas $x_1, \dots, x_d$
Sinapsis	Los pesos $w_1, \dots, w_d$
Soma	La suma ponderada más el sesgo
Axón	La salida, pasada por una función de activación

O sea

$$y = g\left(\sum_{i=1}^d w_i x_i + w_0\right),$$

con g la función de activación.

Idea clave

Con g el signo, esa ecuación es exactamente el perceptrón de la clase del 27 de agosto. La neurona artificial no es un modelo nuevo, es el mismo clasificador lineal contado desde la biología. Lo nuevo viene de conectar muchas en capas, que es donde deja de ser lineal.